Filters vs. Kernels

1 The Kernel — The 2D Weight Matrix

A kernel is a single, two-dimensional grid of numbers (typically odd sizes like 3×3, 5×5, or 7×7). Each cell contains a mathematical value called a weight.

A kernel operates exclusively on a single channel of spatial data (e.g., just the grayscale channel of an image, or just the red channel of an RGB image).

2 The Filter — The 3D Block

A filter is a three-dimensional collection of one or more kernels stacked together. The depth of a filter must always exactly match the number of channels in the input data.

While a kernel is a component, a filter is the complete operational unit applied to an input image.

Spatial Processing of Color (RGB) Images

When a color image is processed, it contains three channels: Red, Green, and Blue.

R — To filter this image, we use one 3D filter containing three distinct 2D kernels — one for each color channel.

G — Each kernel independently slides across its assigned channel, calculating local products.

B — The individual outputs from all three channels are summed together at each pixel location.

Σ — This produces a single, unified 2D output grid called a Feature Map.

Cross-Correlation vs. Convolution

Both operations involve sliding a kernel across an image to calculate a weighted sum of local pixels, but they handle the orientation of the kernel differently.

1. Cross-Correlation

In cross-correlation, the kernel is placed directly over the image patch as-is, without any spatial modification. Overlapping elements are multiplied together, and all products are summed.

$$\text{Output}(x, y) = \sum_{i=-1}^{1} \sum_{j=-1}^{1} f(x+i,\, y+j) \cdot w(i, j)$$

2. Convolution

In true mathematical convolution, the kernel is flipped 180° (inverted both horizontally and vertically) before it is aligned with the image patch. It then follows the same multiply-and-sum process as correlation.

$$\text{Output}(x, y) = \sum_{i=-1}^{1} \sum_{j=-1}^{1} f(x-i,\, y-j) \cdot w(i, j)$$

Why Convolution Dominates Mathematics and AI

Mathematical Properties

Convolution obeys the associative and commutative properties. Cross-correlation does not. These properties allow mathematicians and frameworks to chain, invert, and optimize complex mathematical operations efficiently.

The AI Shortcut

$$(A * B) * C = A * (B * C)$$

$$A * B = B * A$$

Smoothing / Blurring Kernels (Low-Pass Filters)

By configuring the spatial arrangement of positive and negative weights, kernels are tuned to manipulate high-frequency and low-frequency components of an image.

These filters act as low-pass filters, blocking rapid pixel intensity changes (noise, sharp edges) and letting smooth, low-frequency variations pass.

Weight Profile

Every weight is a positive fraction. The sum of all elements must equal exactly 1.0 — this ensures the processed image maintains its original overall brightness.

Gaussian Blur

Assigns maximum weight to the center pixel, smoothly decaying outward via a bell-curve distribution. This yields a much more natural, organic blur without square artifact patterns.

Box Blur (3×3)

Evenly distributes weight across all neighbors

$$\begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

Sharpening Kernels (High-Pass Filters)

These filters amplify high-frequency changes, making boundaries and micro-textures look crisp.

Weight Profile

A powerful positive value is placed in the absolute center, surrounded by negative weights. The sum of all elements equals exactly 1.0, preserving background lighting while exaggerating local differences.

Mechanics

If a center pixel is identical to its neighbors, the negatives balance out the positive, leaving it unchanged. If the center pixel is slightly brighter than its neighbors, the negative surrounding weights amplify that difference dramatically.

Standard Sharpen

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Edge Detection Kernels (Gradient Filters)

These kernels measure spatial gradients — the rate of change of pixel intensity along a specific direction.

Weight Profile

The positive and negative weights are perfectly balanced, summing to exactly 0.0. In solid or flat color zones, the kernel outputs a value of 0 (complete darkness / no edge).

Prewitt Filter (Vertical Edges)

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

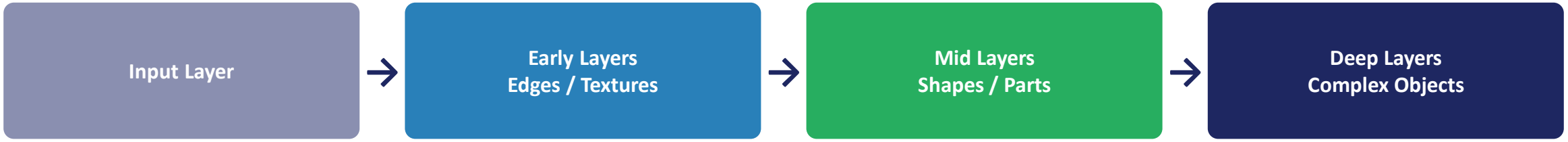
Sobel Filter (Vertical Edges)

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

Refines the Prewitt filter by increasing the weight of the central row to reduce noise vulnerability.

Learned AI Kernels in CNNs

For decades, computer vision relied on specialists manually tuning these matrices. Convolutional Neural Networks (CNNs) inverted this paradigm completely.



Automatic Parameter Optimization

Instead of using hand-crafted, predefined weights, AI networks initialize their kernels with random numbers. As the network looks at thousands of training images, it evaluates its errors and uses an optimization algorithm to subtly rewrite the weights.

Hierarchical Feature Extraction

Early Layers: basic edge detectors, blurring matrices, and color gradient filters.

Middle Layers: combine primitive edge responses into geometric shapes, textures, corners, and simple patterns.

Deep Layers: aggregate shapes into sophisticated concepts — wheels, eyes, text fragments, or complete faces.

Image Blending & Contrast/Brightness Adjustment

Two core techniques for combining and enhancing images with OpenCV

01

What Is Image Blending?

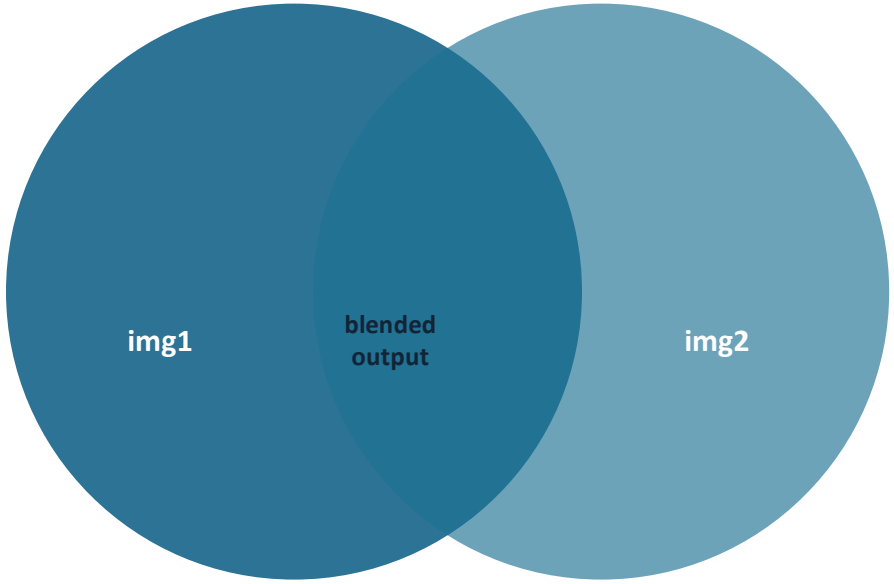
Image blending combines two or more images into a single output by mixing pixel values, typically through a weighted combination — creating a smooth fusion rather than a simple overlay.

Linear (Alpha) Blending Formula

$$dst = \alpha \cdot img1 + \beta \cdot img2 + \gamma$$

```
cv2.addWeighted(img1, alpha, img2, beta, gamma)
```

- **α** weight of the first image
- **β** weight of the second image
- **γ** scalar added to each pixel sum



Blending mixes both images across the overlap — the result is a smooth transition, not a hard cut.

The parameters **α** (Alpha), **β** (Beta), and **γ** (Gamma) have specific ranges and behaviors to ensure the final image looks natural and correctly exposed.

1. α (Alpha) and β (Beta) — The Blend Weights

These two weights control the transparency and influence of each input image.

The Normal Range: Both α and β are decimal fractions ranging strictly between **0.0 and 1.0**.

The Golden Rule (α + β = 1.0): In almost all standard blending scenarios, the sum of alpha and beta must equal exactly 1.0.

Why must they equal 1.0?

If the weights add up to more than 1.0, the combined pixel values will quickly exceed the maximum limit of 255, resulting in an overexposed, "blown-out" white image. If they add up to less than 1.0, the resulting image will look unnaturally dark and muddy.

2. γ (Gamma) — The Brightness Offset

Gamma is not a percentage or a fraction; it is a **scalar offset value (integer)** added to every pixel after the images are blended.

The Normal Range: Typically an integer between **-255 and +255**, though in most real-world blending scenarios, it is set to **0**.

Positive Gamma (> 0): Uniformly brightens the final blended image.

Negative Gamma (< 0): Uniformly darkens the final blended image.

02

Where Image Blending Is Used

1 **Panorama Stitching**

Smoothly merging overlapping regions of multiple photos

2 **Transitions & Fades**

Cross-fading between images in video editing and slideshows

3 **Watermarking**

Overlaying a logo transparently onto an image

4 **Augmented Reality**

Blending virtual objects with the real camera feed

5 **Compositing**

Merging foreground and background (e.g. green-screen effects)

6 **HDR Imaging**

Combining multiple exposures into one high-dynamic-range image

03

Contrast & Brightness in OpenCV

Both are adjusted with a simple linear transformation applied to every pixel:

$$new_pixel = \alpha \cdot old_pixel + \beta$$

α

CONTRAST

alpha > 1 increases contrast
0 < alpha < 1 decreases it

β

BRIGHTNESS

beta > 0 brightens
beta < 0 darkens

convertScaleAbs()

```
img = cv2.imread('image.jpg')

alpha = 1.5   # contrast
beta = 30     # brightness

adjusted = cv2.convertScaleAbs(
    img, alpha=alpha, beta=beta
)
```

Automatically clips values to [0, 255] and converts to uint8 — preventing overflow artifacts.

04

Alternative Methods & Quick Reference

Using addWeighted()

```
adjusted = cv2.addWeighted(
    img, alpha, zeros, 0, beta
)
```

Manual (NumPy) — for understanding

```
np.clip(alpha*img.astype(np.float64)
    + beta, 0, 255).astype(np.uint8)
```

Quick Reference

Parameter	Effect	Typical Range
alpha > 1	Increases contrast	1.0 – 3.0
alpha < 1	Decreases contrast	0.0 – 1.0
beta > 0	Increases brightness	0 – 100
beta < 0	Decreases brightness	-100 – 0

Prefer convertScaleAbs() in production — it clips and casts safely; manual NumPy math can overflow silently.

Key Takeaways

1. Image blending mixes pixel values via cv2.addWeighted() — used in panoramas, AR, HDR, and compositing.

2. Contrast and brightness follow new_pixel = α · old_pixel + β, tuned by two parameters.

3. cv2.convertScaleAbs() is the safe, production-ready way to apply both — it clips and casts automatically.